Анализ корпуса: от потока к правилу к контрпримеру

Английский оригинал: CORPUS_ANALYSIS.md.

Эта заметка прослеживает, как три захвата корпуса привели к правилу v2. Каждое число ниже производится кодом этого модуля и воспроизводится командами, названными в конце. Она намеренно скупа на метод и богата на числа, чтобы читатель мог проверить утверждение по байтам.

1. Захваты и их потоки

Корпус — три захвата, все A/B TCP-обмены между одной парой конечных точек. Модуль захвата выгружает каждый как нормализованный JSON, в котором каждое направление каждой сессии — один поток байт, с дырами пересборки, записанными как диагностики, а не заполненными нулями.

захват	capture_id (префикс)	сессий	байт потока
corpus_capture_01.pcapng	sha256:1a2f3637124ab	3	3036
corpus_capture_02.pcapng	sha256:4aa281b97de9a	2	520
corpus_capture_defects.pcapng	sha256:ae93a5b223808	1	74

`corpus_capture_defects` несёт `gap` в `s1 A_to_B` (2 недостающих байта на смещении 19) и `ambiguity` в `s1 B_to_A` (конфликтующее перекрытие на смещении 32, где сохранено `2008`, а `ffff` отвергнуто). Два других захвата чисты.

2. Фрейминг: три кандидата против байтов

Поле длины стоит на смещении 2, два байта, big-endian. Что оно считает, не записано. Три чтения были сфреймлены против каждого потока и сохранены только если поток потреблялся точно, без обрезанного последнего сообщения и без невозможного байта команды.

поток	байт	payload	payload+length	всё сообщение
c01 s1 A_to_B	300	60/300 чисто	1/0 сломано	1/0 сломано
c01 s1 B_to_A	420	60/420 чисто	3/420 сломано	1/0 сломано
c01 s2 A_to_B	420	60/420 чисто	2/420 сломано	1/0 сломано
c01 s2 B_to_A	420	60/420 чисто	2/420 сломано	1/0 сломано
c01 s3 A_to_B	100	20/100 чисто	1/0 сломано	1/0 сломано
c01 s3 B_to_A	1376	20/1376 чисто	3/1376 сломано	2/1376 сломано
c02 s1 A_to_B	100	20/100 чисто	1/0 сломано	1/0 сломано
c02 s1 B_to_A	140	20/140 чисто	2/140 сломано	1/0 сломано
c02 s2 A_to_B	140	20/140 чисто	2/140 сломано	1/0 сломано
c02 s2 B_to_A	140	20/140 чисто	2/140 сломано	1/0 сломано
defects s1 A_to_B	24	4/24 сломано	1/0 сломано	1/0 сломано
defects s1 B_to_A	50	5/50 чисто	2/50 сломано	1/0 сломано

`payload` фреймит все одиннадцать чистых потоков точно и является единственным кандидатом, который это делает. `payload_and_length_field` разбивает поток на несколько завышенных блоков (он потребляет поток, но оставляет байты команды на невозможных значениях), а `entire_message` останавливается после первого размера. Двенадцатый поток, тот с пропуском, — граница применимости: никакой фрейминг не может сфреймить через дыру, поэтому сообщение, которое её пересекает, `incomplete`, а не `matched`. Полную таблицу см. в `docs/REFERENCE_INVESTIGATION.md`.

Число здесь одиннадцать из двенадцати, потому что эта перепись покрывает все три захвата. Генератор фрейминга (`scripts/run_reference_investigation.py`) работает по своему корпусу по умолчанию из двух чистых захватов, где тот же вердикт читается как десять из десяти; два числа различаются только тем, что считают, а не исходом.

3. Раскладка полей, которая выжила

смещение	размер	тип	имя	смысл
0	1	uint8	command	код запроса
1	1	uint8	flags	зарезервировано в каждом наблюдаемом сообщении
2	2	uint16 BE	payload_length	байты после поля длины
4	1	uint8	target	идентификатор параметра
5	2	uint16 BE	value	записанное значение, присутствует только при записи

Раскладка идентична в обоих направлениях на проводе; направление меняет лишь то, какие значения `command` допустимы. Три из смыслов байт — гипотезы (`command`, `target`, `value`), помеченные `hypothesis: true` в правиле, и движок альтернатив прогоняется по всем трём.

4. Правило v1 и восемьдесят контрпримеров

Правило v1 было ограничено `A_to_B` и объявляло `command` единственным значением `0x01`.

проверка	matched	mismatched	incomplete	not_applicable
v1 на corpus_capture_01	60	80	0	140
v1 на corpus_capture_02	20	20	0	40
v1 на synthetic_live	0	0	1	1

Восемьдесят несовпадений в c01 все на смещении 0, в сессиях `s2` (команда `0x02`) и `s3` (команда `0x03`). Каждый привязан к своим байтам, например `02000003130055` (`s2`, первый контрпример), чтобы читатель видел байт, сломавший правило. Распределение чистое и по сессиям:

сессия	виденные байты команды	сообщений
s1	01	60
s2	02	60
s3	03	20

`not_applicable` считает потоки `B_to_A`: область говорит `A_to_B`, поэтому правило ничего не утверждает о другом направлении, и движок записывает это, а не сообщает ложное совпадение.

5. Альтернативы: `0x01` — вся правда?

Перед расширением правила поле отдали движку альтернатив. Для `command` в c01 он сообщает:

чтение	поддержка	противоречий	балл
entropy_enum	140	0	1.00
low_cardinality	140	0	1.00
offset_shift_+1	140	0	1.00
counter	137	2	0.99
constant	60	80	0.43

`constant` (чтение v1) опровергается более чем половиной сообщений; `entropy_enum` подходит ко всем и соглашается, что пространство значений маленькое и замкнутое. Это доказательство за расширением правила, и поэтому шаг 5 шёл перед шагом 6.

6. Правило v2 и сравнение версий

Правило v2 оставляет раскладку и фрейминг нетронутыми и расширяет `command` до `[1, 2, 3]`. Сравнение двух отчётов по одному захвату:

переход	разрешено	внесено
v1 -> v2 на corpus_capture_01	80	0

проверка	matched	mismatched	incomplete	not_applicable
v2 на corpus_capture_01	140	0	0	140
v2 на corpus_capture_02	40	0	0	40
v2 на synthetic_live	0	0	1	1

Все восемьдесят контрпримеров разрешены единственным изменением формы, и ни один не внесён. Старый результат v1 помечается `outdated`; он никогда не показывается как текущий вердикт.

7. Где правило останавливается

Расширение `command` до значений корпуса не делает правило общим. На `synthetic_live` то же правило не совпадает ни с чем. Захват использует little-endian длину, поэтому при big-endian длине правила оба потока обрезаны: ни один не фреймится чисто, и ограниченное правило сообщает одно `incomplete` сообщение на `A_to_B`, а остальные `not_applicable`. Правило падает на поле длины прежде, чем судится любой байт команды, и говорит об этом, а не гадает. Это граница применимости, ради раскрытия которой построено всё упражнение: правило подтверждается внутри байтов, против которых проверялось, а не доказывается за их пределами.

8. Шесть статусов, каждый со свидетелем

Классификация — не список слов в документе; каждый статус производится движком из поставляемых захватов. `tests/integration/test_status_coverage.py` падает, если любой из них перестаёт быть достижим.

статус	свидетель на корпусе	правило и поток
`matched`	60 запросов, чья команда в объявленном наборе	правило v1, `corpus_capture_01` `s1` `A_to_B`
`mismatched`	80 запросов с командой вне {1}	правило v1, `corpus_capture_01` `s2` + `s3` `A_to_B`
`incomplete`	сообщение, чей фрейминг достигает пропуска	неограниченное правило на `corpus_capture_defects` или поле за концом сообщения
`ambiguous`	сообщение, чей диапазон поля покрывает конфликтующее перекрытие	неограниченное правило на `corpus_capture_defects` `B_to_A` (смещение 32)
`uncovered`	сообщение, чьё единственное объявленное поле — необязательное и отсутствует	необязательное хвостовое поле на `corpus_capture_01`
`not_applicable`	каждый ответ, который ограниченное правило не интерпретирует	правило v1, `corpus_capture_01` `B_to_A`

`outdated` и `unknown` стоят вне основных шести: `outdated` — метка версионирования на замещённом результате (раздел 6), а не чтение байтов, а `unknown` — значение движка по умолчанию до применения любого правила.

9. Как воспроизвести

```
python -m pytest tests/integration/test_reference_report.py -q
python -m src.protocol.cli verify --rule examples/corpus_rule_v1.json \
    --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
    --out /tmp/v1.json
python -m src.protocol.cli alternatives --rule examples/corpus_rule_v1.json \
    --report /tmp/result.json --out /tmp/alternatives.json
```

Перепись фрейминга производится `scripts/run_reference_investigation.py`.

10. Набор организаторов

Этот анализ использует синтетический корпус, потому что он намеренно упражняет выборы фрейминга. Организатор предоставил второй, независимый набор захватов для протокола «термостенд»; он закоммичен в `tests/corpus/real/` и изучен в `docs/REAL_CORPUS_ANALYSIS.md`. Он прогоняет тот же цикл против другого протокола (магия `0xa65c`, поле длины, считающее заголовок): наивное правило оставляет шесть контрпримеров, правило со смещением заголовка их убирает, и все 90 сообщений по четырём захватам совпадают с полным покрытием байт. Синтетический корпус остаётся эталоном для обсуждения фрейминга; реальный корпус — проверка применимости.

```
python -m scripts.investigate_dataset
python -m pytest tests/hypothesis/test_real_corpus.py -q
```